



Optimized Deep Isolation Forest

Łukasz Gałka 

Department of Computational Intelligence, Lublin University of Technology, Nadbystrzycka 36B, 20-618, Lublin, Poland

ARTICLE INFO

Editor: Jiwen Lu

Keywords:

Anomaly detection
Isolation Forest
Deep Isolation Forest
Neural networks
Outlier detection

ABSTRACT

Anomaly detection and the identification of elements that do not fit the data characteristics are increasingly used in information systems, both for data cleaning and for finding unusual elements. Unsupervised anomaly detection methods are particularly useful in this context. This paper introduces the Optimized Deep Isolation Forest (ODIF) as an optimized version of the Deep Isolation Forest (DIF) algorithm. The training of DIF is subjected to an optimization of the operations performed, which leads to a reduction of the computational and memory complexity. In a series of experiments, both DIF and ODIF are implemented, and their effectiveness is evaluated using Area Under the Precision-Recall Curve (PR AUC). The proposed method demonstrates significantly better detection performance compared to the baseline Isolation Forest and competitive techniques. Additionally, the execution times of the training phase are measured for both the CPU and GPU stages, as well as memory usage, including RAM and VRAM. The results unequivocally indicate a much faster execution of the ODIF algorithm compared to DIF, with average CPU stage and GPU stage times being over one and a half times and nearly 150 times shorter, respectively. Similarly, memory usage is significantly reduced for ODIF in comparison to DIF, with RAM consumption lowered by approximately 18% and VRAM by over 55%.

1. Introduction

Anomaly detection (AD) is one of the most critical challenges in today's information systems. AD algorithms are used both for data cleaning and for analyzing and identifying elements that do not fit the overall characteristics of the data. These methods can be categorized based on the availability of data labels during the training phase, namely into supervised and unsupervised solutions. AD techniques are widely applicable across numerous domains, including cybersecurity, fraud prevention and identification, medical diagnostics, and industrial systems for fault and defect detection. Additionally, they are essential in various other contexts where data integrity and reliability are critical. Developing AD algorithms that are accurate, scalable, and resource-efficient remains a significant challenge and is a highly sought-after objective in contemporary data-driven environments.

The literature highlights a variety of techniques used for AD. Firstly, statistical computation-based solutions should be noted. These include methods such as various probabilistic models, Gaussian processes, and Hidden Markov models [1]. Distance-based methods like two-class k-nearest neighbor [2] and density-based methods such as weighted neighborhood density estimation [3] are also commonly used. Anomalous elements can be associated with clustering, where anomalies may either form small clusters or appear at the edges of clusters [4]. In such cases, clustering analysis methods like *k*-means, hierarchical clustering, or DBSCAN are applied [5]. For detecting anomalies in non-linear dependency modeling, support vector regression, an extension

of support vector machine for regression problems, is introduced [6]. Anomalies are detected by replacing the Euclidean distance with other dissimilarity measures, such as the fuzzy Gaussian dissimilarity measure [7]. Ensemble methods such as Random Forest, which extend decision tree-based approaches to improve robustness and accuracy, are particularly effective in anomaly classification [8]. Expert systems based on rules and incorporating domain-specific expert knowledge are also developed [9]. Solutions based on various types of neural networks are gaining increasing popularity. These include deep neural networks, recurrent neural networks, long-short term memory neural networks, convolutional neural networks, autoencoders, spiking neural networks, and generative adversarial networks [10,11]. Finally, it is important to mention hybrid algorithms that combine multiple techniques [12].

Among the wide range of unsupervised anomaly detection algorithms, particular attention has been given to tree-based ensemble methods due to their efficiency and scalability. One of the most popular methods based on isolation trees concept is the Deep Isolation Forest (DIF) [13]. This method introduces the transformation of data samples into new representation spaces and then forms isolation forests for these representations. The solution is particularly significant and effective in detecting hard anomalies, where other algorithms fail to effectively separate such samples. It is crucial to emphasize that anomaly detection, especially unsupervised, often requires fast processing times. Therefore, the process of constructing and evaluating the DIF method is analyzed. It is found that it is possible to optimize the training phase operations of this algorithm without compromising its effectiveness,

E-mail address: l.galka@pollub.pl.

<https://doi.org/10.1016/j.patrec.2025.07.014>

Received 15 September 2024; Received in revised form 25 April 2025; Accepted 14 July 2025

Available online 25 July 2025

0167-8655/© 2025 The Author. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

leading to faster execution and reduced resource usage. Specifically, the data transformation process and subsequent isolation tree construction can be segmented and enhanced. Such approach results in a significant reduction in the number of samples subjected to the transformation and representation creation processes.

The main contributions of this study are as follows:

1. The design of the Optimized Deep Isolation Forest algorithm, which incorporates a novel training scheme aimed at optimizing operations performed, results in a reduction of both the execution time and memory complexity of the original DIF method.
2. Experimental validation of the proposed solution on a collection of 18 real-world datasets, assessing effectiveness (using PR AUC), training phase execution time (CPU and GPU), and memory usage (RAM and VRAM).

The rest of the manuscript is organized as follows. Section 2 presents related works concerning isolation forests and their extensions. Section 3 recalls the original DIF method. Section 4 describes the novel proposal for optimizing the DIF method. Section 5 is dedicated to the results of numerical experiments. Finally, Section 6 summarizes the work and outlines directions for future research.

2. Related works

This manuscript focuses on the DIF algorithm, an enhancement of the Isolation Forest (IF) method [14,15], which remains one of the fastest and most effective approaches for anomaly detection. The literature review presents methods that are considered competitive with DIF, specifically advanced variants derived from the original IF. This provides a broader context for evaluating the proposed solution against related isolation based approaches.

The IF method relies on recursive random partitions of the data space according to a random attribute, creating so-called isolation trees. Improvements include random rotating the data space to avoid axis-parallel splits [16]. Additionally, the random selection of split values in isolation nodes is replaced by finding the optimal split value [17]. The data space is also transformed using kernel principal component analysis, preparing the data for training isolation trees with only k principal components [18]. Another approach employs transforming the data using a sparse random matrix [19]. Modifications also involve the evaluation function while retaining the original tree construction by replacing the average path length evaluation with direct estimation of mixture coefficients [20]. The isolation forest is also combined with clustering methods, particularly when the data is grouped into clusters [21]. In [22], the authors introduce a new manner of forming isolation trees based on the minimal spanning tree algorithm. This concept is further extended in [23] by applying the Takagi-Sugeno fuzzy inference model in the anomaly score evaluation function. To enhance the method's effectiveness, local isolation forests are constructed based on layers formed on the basis of equally sized subspaces [24]. Generalized sample separation is applied based on pairwise proximity of randomly selected pairs in the isolation node [25]. Particularly significant are algorithms incorporating neural networks into the IF mechanism. Here, deep neural networks are used to transform data into several representations, creating isolation trees for each representation [13]. Moreover, tree structure optimization models are built using deep neural networks with genetic algorithms [26]. Noteworthy are algorithms that use hashing functions for element separation, such as order-preserving hashing [27]. A recent solution worth mentioning involves deterministic attribute selection in isolation nodes while maintaining randomness in value selection [28]. Interested readers can find an in-depth review of IF algorithm modifications in [29,30].

In addition to algorithmic enhancements, numerous studies have demonstrated the practical applicability of mentioned methods across various domains. For instance, in [31], an IF-based framework is applied to monitor energy consumption in industrial buildings. In the

field of geosciences, [32] employed DIF to detect anomalous regions in geological datasets for the purpose of locating potential mineral deposits. In the context of cybersecurity, IF variants have been successfully used to enhance the safety of Industrial Control Systems (ICS) [33]. Similarly, [34] explored the use of mentioned techniques to protect in-vehicle CAN bus communication systems. Furthermore, in the domain of healthcare, [35] introduced an IF-based solution for real-time health monitoring using low-power hardware such as Arduino. These examples underline the versatility and effectiveness of IF-based methods in diverse real-world scenarios, reinforcing their relevance not only in theoretical research but also in practical deployments.

3. Deep isolation forest

In this section, the basic Deep Isolation Forest algorithm (DIF) [13] is recalled. The method introduces representations of samples using a deep neural network. Samples are transformed into their representations in a new data space through the mentioned neural network. As a result, the partitions in the new data space create nonlinear splits in the original data space, replacing axis-parallel partitions. Each sample has r representations to isolate hard-to-detect anomalies.

The data transformation to the new space uses the Computational-Efficient deep Representation Ensemble (CERE) algorithm. This algorithm relies on a single neural network that transforms sample attributes into their r representations. The network is designed to compute all representations simultaneously. Notably, the network does not require training, as its parameters are randomly assigned. Specifically, each layer of the network computes r representations using random base weights, denoted as $W_0 \in \mathbb{R}^{a \times b}$, and two small vectors, $r_i \in \mathbb{R}^a$ and $s_i \in \mathbb{R}^b$, where i indicates the representation number, a denotes the number of input dimensions of the layer, and b denotes the number of output dimensions of the layer, with x being the layer input. This transformation for the i th representation is described as

$$W_i^T x = (W_0^T (x \circ r_i)) \circ s_i \quad (1)$$

where $\circ$ denotes the Hadamard product.

Thus, the computations of all representations in the network layer are described as follows

$$\begin{bmatrix} XW_1 \\ XW_2 \\ \vdots \\ XW_r \end{bmatrix} = \left(\begin{bmatrix} X \\ X \\ \vdots \\ X \end{bmatrix} \circ \begin{bmatrix} R_1 \\ R_2 \\ \vdots \\ R_r \end{bmatrix} \right) W_0 \circ \begin{bmatrix} S_1 \\ S_2 \\ \vdots \\ S_r \end{bmatrix} \quad (2)$$

where $X \in \mathbb{R}^{c \times a}$ is a mini-batch of data with size and dimensionality equal to c and a , respectively. Each row of $R_i \in \mathbb{R}^{c \times a}$ contains the repeated vector r_i , and similarly, each row of $S_i \in \mathbb{R}^{c \times b}$ contains the repeated vector s_i .

This network layer construction enables efficient computation of r representations using a single neural network. It is worth mentioning that the remaining computations of the network layer and the entire network remain the same as in the base network, i.e. the layers are processed sequentially, and each layer applies an activation function after multiplying the vector by the weights.

After the initial transformation of samples into their representations, isolation forests are built for each representation. The IF method is used in its original form [14]. Specifically, for each i th representation, a forest consisting of t trees is created by randomly sampling n samples from the i th representation sample set. Thus, each representation has a single isolation forest. By default, the DIF training phase is invoked with the hyperparameters: $r = 50$ (number of representations), $c = 64$ (batch size), $t = 6$ (number of trees per representation), and $n = 256$ (number of samples per isolation tree).

To enhance the understanding of the DIF training procedure, Fig. 1 presents a graphical illustration of the formation process. Each sample is transformed into r representations using the CERE mechanism. For each representation, an Isolation Forest with t trees is constructed.

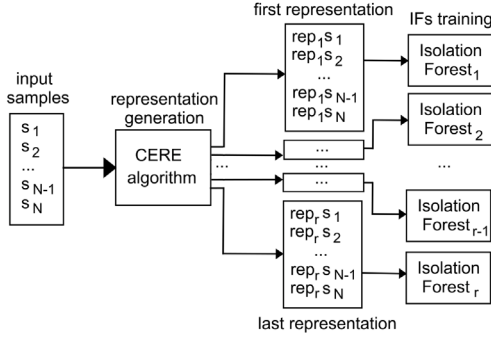


Fig. 1. Visualization of the training process of DIF.

After the DIF method is constructed, the algorithm is ready for element evaluation. At this stage, the Deviation-Enhanced Anomaly Scoring function (DEAS) is introduced. Specifically, the evaluation of an element for a single isolation forest is based on two components. The first component is related to the path length while traversing the isolation tree. The second component is associated with the deviation degree for the traversed path, calculated as the average distance between the sample attribute value and the split value in the traversed isolation nodes. Firstly, the sample is transformed into r representations using the neural network prepared in the training phase. For each representation, an anomaly score is prepared by traversing the isolation tree nodes with the sample representation, similar to the IF method. The anomaly score for the sample and a given representation is described as follows

$$score(x_{rep_i}, n) = 2^{-\frac{E(h(x_{rep_i}))}{us(n)}} \cdot F(d(x_{rep_i})) \quad (3)$$

where x_{rep_i} is the i th considered representation of the analyzed sample, n is the number of samples forming a single isolation tree, $E(h(x_{rep_i}))$ is the average path length when traversing trees created for the i th representation of the element x_{rep_i} , and $F(d(x_{rep_i}))$ is the average deviation degree obtained when traversing trees created for the i th representation of the element x_{rep_i} . More

$$us(n) = 2(\ln(n-1) + 0.5772156649) - \frac{2(n-1)}{n} \quad (4)$$

The final anomaly score for a sample using the DIF method is the average of the scores for the individual representations.

In the last part of this section, the computational and memory complexity of the DIF algorithm is presented. The overall time complexity of the method is $O(Nar + rt \log n)$, where N denotes the number of training samples, a is the samples dimensionality, r is the number of representations, t is the number of isolation trees per representation, and n is the number of samples per tree. The memory complexity is primarily determined by two components: $O(Nrb)$ for storing the transformed representations and $O(rt \log n)$ for storing the isolation forests for each representation, where b denotes the output dimensionality of a single representation.

4. Optimized deep isolation forest

This section describes the optimization method for the training phase of the DIF algorithm, referred to as Optimized Deep Isolation Forest (ODIF). The second phase of the algorithm remains unchanged, ensuring that the anomaly evaluation of elements proceeds as in the base method.

As can be easily observed, the original DIF method in the training phase transforms samples into their representations in new data spaces. Subsequently, the IF is constructed based on sampling from the prepared representations. It should be emphasized that when the

number of training samples is large, all samples undergo transformation, leading to increased computational complexity and memory usage. This process can be optimized. Not all samples are used in the tree formation process, only those sampled for a given tree. Hence, the idea is to sample the elements to be used in the tree formation process before transforming them into their representations. Only those samples selected for future tree formation are passed for representation generation. This significantly reduces the number of samples that need to be transformed. The transformation process is then applied to this reduced sample set. To ensure that samples are not resampled during the IF construction phase, the forest formation process is modified to eliminate subsampling at this stage.

The optimized training phase algorithm is presented in Algorithm 1. The training begins by determining the total number of samples required for the tree formation process. This is done by multiplying the number of trees per representation by the number of samples per tree. The algorithm then randomly selects samples indices. This sampling is performed upfront to decide which samples should be used for tree construction, thereby reducing the number of samples that need to be transformed into new representations. Next, the algorithm handles redundant samples that might result from the random sampling process. It extracts unique indices, eliminating duplicates to ensure that each sample is transformed only once. Additionally, it creates inverse indices, which is a mapping that is used later to reference the unique samples during the formation of isolation trees. The algorithm then extracts the corresponding samples and stores them in subset. This subset contains only the samples that are used in the transformation and tree formation process, significantly reducing the computational load. For each sample in the subset, the algorithm applies the CERE mechanism to generate r different representations. This step is crucial as it generates the new data spaces in which the isolation trees are constructed. By applying the transformation only to this subset of samples, the algorithm achieves a substantial reduction in computational complexity and memory usage. Following the generation of representations, the algorithm proceeds to construct r isolation forests, each corresponding to one of the r representations. For each forest, it initializes an empty list of forests to hold the isolation trees. It then creates each tree by collecting pre-sampled elements. For every tree in the forest, an empty list of samples is created to store the relevant samples. The algorithm uses mapping to retrieve the indices of the unique samples. Subsequently, a modified isolation tree is created based on the collected samples, without additional subsampling. This step ensures that the samples selected in the initial phase are the only ones used in tree formation, avoiding the computational overhead of subsampling. The constructed tree is then added to the current forest. Once all trees for a representation are created, the completed forest is appended to the list of forests. Importantly, this change does not alter the structure of the resulting isolation forests, ensuring that the anomaly detection capabilities of the original DIF algorithm are preserved.

The computational and memory complexity of the ODIF algorithm is reduced compared to the DIF method. Unlike DIF, ODIF transforms only a subset of nt samples, where t is the number of trees per representation and n is the number of samples per tree. As a result, the overall time complexity is reduced to $O(nar + rt \log n)$, where a denotes the samples dimensionality and r is the number of representations. The memory complexity is dominated by two components: $O(nrb)$ for storing the transformed representations, and $O(rt \log n)$ for storing the isolation forests, where b is the output dimensionality of a single representation. These reductions make ODIF more suitable for scenarios involving large datasets or limited computational resources.

5. Numerical experiments

This section presents the results of numerical experiments. The experiments are conducted on a laptop with the following technical specifications: Intel Core i7–10750H processor, GTX 1660 Ti graphics

Algorithm 1 ODIF: Optimized training phase

Inputs:

- X – input samples
- r – representations number
- t – trees number per representation
- n – samples number per tree

$samples_number \leftarrow t \cdot n$
 $indices \leftarrow$ randomly select $samples_number$ sample indices for the formation of the IF
 $unique_indices \leftarrow unique(indices)$
 $inverse_indices \leftarrow$ create a list mapping indices from $indices$ to the list $unique_indices$
 $X_{subset} \leftarrow \emptyset$
for $i \leftarrow 1$ to $|unique_indices|$ **do**
 X_{subset} append $X[unique_indices[i]]$
end for
 $X_{rep} \leftarrow \emptyset$
for $i \leftarrow 1$ to $|X_{subset}|$ **do**
 $temp \leftarrow$ generate a list of r representations via CERE
 X_{rep} append $temp$
end for
 $forests \leftarrow \emptyset$
for $i \leftarrow 1$ to r **do**
 $iForest \leftarrow \emptyset$
 for $j \leftarrow 1$ to t **do**
 $samples \leftarrow \emptyset$
 for $k \leftarrow 1$ to n **do**
 $idx \leftarrow inverse_indices[j \cdot n + k]$
 $samples$ append $X_{rep}[idx][i]$
 end for
 $iTree \leftarrow$ create a modified isolation tree based on $samples$ without subsampling
 $iForest$ append $iTree$
 end for
 $forests$ append $iForest$
end for

card, 32 GB RAM, and 6 GB graphics card memory. All measurements are repeated one hundred times for non-deterministic results, and the presented values are averages of these measurements with standard deviations given in parentheses. The optimization focused on the training phase of the algorithm, and measurements for this phase are presented. In order to position the proposed method within the broader context of unsupervised anomaly detection, a comparative analysis is conducted against the DIF, the original IF, as well as two competitive non-IF-based techniques: Empirical Cumulative Distribution-based Outlier Detection (ECOD) [36] and Score-Guided AutoEncoder (SGAE) [37]. All algorithms are implemented in Python using the PyTorch library and default hyperparameters to ensure a consistent experimental setup. The source code is available in the repository: <https://github.com/lukasz-galka-algorithms/optimized-deep-isolation-forest>. To assess the detection performance of all methods, the Area Under the Precision-Recall Curve (PR AUC) is used as the efficiency evaluation metric. In addition, execution times and memory usage during the training phase are measured for operations performed on both the CPU and GPU. CPU and GPU execution time and memory usage measurements are performed using single CPU thread. Execution times are recorded with a high-resolution monotonic timer, based on start and stop timestamps of each code segment. For memory usage monitoring, a dedicated background thread is launched, continuously sampling the memory footprint of the algorithm execution process with a 1 ms resolution. RAM usage is recorded via the psutil library by tracking the peak resident set size (RSS), and VRAM is tracked using NVIDIA's NVML interface through the pynvml API. In both memory usage cases, the maximum observed value is reported as the final result. In cases where a method does not utilize GPU resources, the corresponding results

Table 1

Details of used datasets.

Dataset	Observations	Dimensionality	Anomalies
ALOI	49 534	27	1508 (3.0%)
Backdoor	95 329	196	2329 (2.4%)
Fault	1941	27	673 (34.7%)
Fraud	284 807	29	492 (0.2%)
Glass	214	9	9 (4.2%)
Ionosphere	351	33	126 (35.9%)
Landsat	6435	36	1333 (20.7%)
Letter	1600	32	100 (6.2%)
Mnist	7603	100	700 (9.2%)
PageBlocks	5393	10	510 (9.5%)
Satellite	6435	36	2036 (31.6%)
Satimage-2	5803	36	71 (1.2%)
Skin	245 057	3	50 859 (20.8%)
Smtip	95 156	3	30 (0.0%)
Vowels	1456	12	50 (3.4%)
Waveform	3443	21	100 (2.9%)
Wilt	4819	5	257 (5.3%)
Wine	129	13	10 (7.8%)

are marked as “not applicable” (n/a). To ensure a fair comparison, all configuration parameters in ODIF match those of the original DIF implementation. A Multilayer Perceptron Network is specifically used to implement the CERE mechanism.

5.1. Datasets details

In order to compare various execution scenarios, the methods are tested on real-world anomaly detection datasets from the Anomaly Detection Benchmark (ADBench) framework [38]. Collections with diverse sample sizes and dimensionalities are selected, and the details of the chosen datasets are presented in Table 1.

5.2. Performance analysis

The results of the PR AUC metric are presented in Table 2. In comparisons between ODIF and DIF, the values are close, which confirms that the optimization preserves the detection effectiveness of the original method. The observed differences are primarily due to the non-deterministic nature of the evaluated methods. ODIF consistently outperforms the original IF across all datasets, and in the vast majority of cases, it also achieves superior results compared to ECOD and SGAE. These findings are further supported by the mean and median PR AUC scores aggregated over all datasets, which clearly indicate the overall advantage of ODIF over the competing approaches. To validate the statistical significance of the observed differences, Wilcoxon signed-rank tests are conducted at a significance level of $\alpha = 0.05$ to compare ODIF against each of the remaining algorithms. The resulting p -values are reported in Table 2. As expected, there is no statistically significant difference between ODIF and DIF, which supports the conclusion that DIF can be reliably replaced with its optimized counterpart. Moreover, the comparisons between ODIF and IF, ECOD, and SGAE reveal that ODIF achieves statistically significantly better performance in all cases.

5.3. Time measurements

The second measured parameter is the execution time of the training phase for two stages, i.e., operations performed on the CPU and operations performed on the GPU. The measurement results for all methods, namely ODIF, DIF, IF, ECOD, and SGAE, are presented in Table 3, while Fig. 2 provides a visualization of the results for ODIF and DIF. In order to assess the statistical significance of the observed differences, Wilcoxon signed-rank tests are conducted, as in the previous evaluation. Again, the significance level is set to $\alpha = 0.05$.

The analysis begins with the CPU stage execution times in the comparison between ODIF and DIF. ODIF exhibits lower execution

Table 2

PR AUC performance results for ODIF, DIF, IF, ECOD, and SGAE, along with p -values from statistical tests comparing ODIF with the other methods.

Dataset	Algorithm				
	ODIF [%]	DIF [%]	IF [%]	ECOD [%]	SGAE [%]
ALOI	4.7 (0.2)	4.6 (0.2)	3.3 (0.0)	3.3	3.4 (0.0)
Backdoor	41.1 (5.4)	41.2 (3.7)	5.3 (1.6)	9.2	52.2 (5.2)
Fault	49.4 (2.4)	49.9 (2.0)	41.7 (1.2)	32.5	35.7 (5.4)
Fraud	38.3 (3.8)	38.1 (3.7)	14.8 (3.8)	21.5	21.9 (16.6)
Glass	12.3 (4.2)	12.1 (3.8)	7.1 (1.1)	8.0	7.7 (7.8)
Ionosphere	89.4 (2.2)	92.3 (1.1)	79.4 (0.9)	65.5	51.1 (14.4)
Landsat	25.5 (1.0)	25.4 (1.0)	19.6 (0.9)	16.3	19.9 (2.9)
Letter	10.3 (1.6)	10.1 (1.4)	8.6 (0.5)	7.5	6.9 (1.4)
Mnist	51.7 (3.8)	51.5 (3.4)	28.9 (3.9)	17.7	43.0 (3.2)
PageBlocks	71.1 (3.3)	72.0 (2.6)	46.5 (1.5)	51.9	35.9 (11.1)
Satellite	72.8 (1.3)	72.7 (1.4)	66.4 (1.7)	52.6	67.0 (5.4)
Satimage-2	93.6 (1.9)	94.2 (1.1)	91.7 (1.2)	65.8	29.0 (33.9)
Skin	32.4 (1.3)	31.9 (1.6)	25.3 (1.1)	18.2	33.8 (9.1)
Smtip	55.9 (8.1)	57.2 (7.0)	0.4 (0.0)	47.6	21.1 (15.5)
Vowels	22.6 (8.0)	25.3 (8.5)	13.5 (4.1)	7.4	5.1 (5.4)
Waveform	9.2 (1.8)	9.2 (1.6)	5.6 (0.6)	3.9	3.8 (1.2)
Wilt	7.1 (0.6)	7.0 (0.5)	4.4 (0.2)	4.1	4.6 (1.3)
Wine	23.1 (12.8)	23.1 (13.9)	19.2 (3.4)	16.6	12.9 (14.6)
Average:	39.5 (3.5)	39.9 (3.3)	26.8 (1.5)	25.0	25.3 (8.6)
Median:	35.3 (2.3)	35.0 (1.8)	17.0 (1.2)	17.2	21.5 (5.4)
p-value:	–	0.5226	0.0000	0.0000	0.0007

times in the majority of datasets, as confirmed by the average values, where ODIF achieves approximately one and a half times faster processing than DIF. The performance gap becomes especially pronounced for high-volume datasets, where several-fold reductions in training time are observed. Moreover, increasing data dimensionality leads to prolonged execution times due to the larger volume of information processed. It is important to highlight that ODIF achieves statistically significantly shorter CPU stage durations. In comparison with the SGAE method, which leverages neural networks, no statistically significant difference is observed in CPU execution times. Nonetheless, the average training time of SGAE is substantially higher. Other methods, such as IF and ECOD, demonstrate faster CPU performance. However, ECOD shows a noticeable slowdown for more extensive datasets. The CPU stage in the experiments relies on single-threaded computation. Hence, multi-threaded implementation would likely reduce execution times. Despite this, ODIF consistently maintains CPU execution times below one second, offering an advantage particularly relevant for real-time processing scenarios.

The second time-related measurement involves the GPU stage. In this phase, IF and ECOD do not utilize GPU computations. Among the remaining methods, ODIF consistently achieves the shortest GPU execution times, averaging approximately 20 ms. This observation is corroborated by p -values obtained from statistical testing, indicating that ODIF performs significantly faster than both DIF and SGAE. DIF records considerably longer execution times due to the need for processing a larger number of samples to build representations. SGAE demonstrates the slowest GPU performance, primarily due to the additional overhead of training an autoencoder, a step that is not required in DIF and ODIF. Notably, ODIF is approximately 150 times faster than the baseline DIF method in GPU stage execution, emphasizing its suitability for time-sensitive and performance-critical applications.

5.4. Memory usage

The last parameter considered is memory usage, encompassing both RAM and VRAM. It is crucial to emphasize that the amount of memory used is key, as it indicates the potential applicability of the method in limited resource scenarios. The measurements of the maximum memory usage during the training phase are presented in Table 4 for all analyzed methods, while Fig. 3 visualizes the results for ODIF and DIF. Table 4

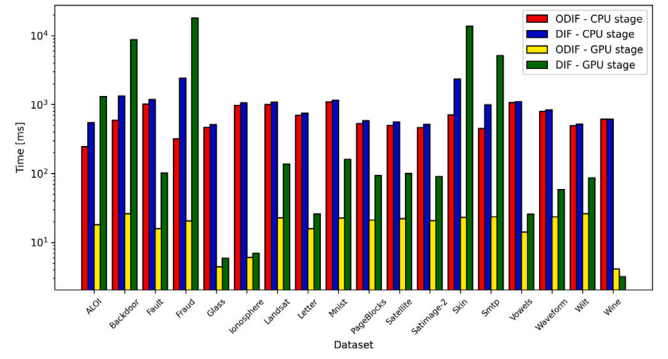


Fig. 2. Train times for CPU and GPU stages of DIF and ODIF methods for each dataset.

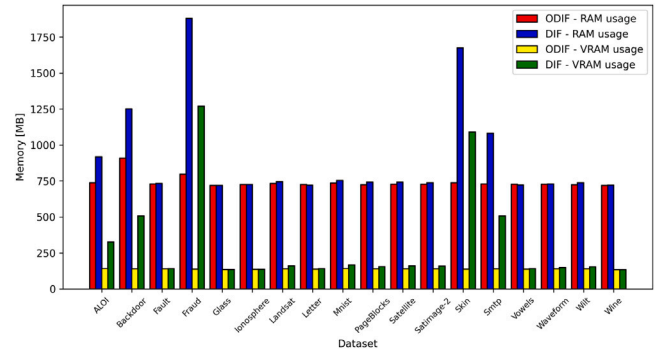


Fig. 3. Train memory usage for CPU and GPU stages of DIF and ODIF methods for each dataset.

also includes the results of statistical tests conducted analogously to the previous parameters, with the significance level set to $\alpha = 0.05$.

The comparison begins with RAM consumption. All evaluated methods exhibit usage within the range of several hundred megabytes. When comparing ODIF and DIF, ODIF achieves, on average, approximately 18% lower RAM usage across all datasets. This difference is statistically significant. ODIF performs especially well on large datasets, where DIF shows a noticeable increase in RAM consumption. For datasets with fewer training samples, the results are comparable. In relation to the remaining methods, IF and ECOD require less RAM than ODIF, while SGAE demonstrates the highest memory demands among all algorithms. Overall, ODIF maintains consistent RAM usage regardless of the training dataset size, making it a favorable option in large-scale data scenarios.

The analysis now turns to VRAM consumption during GPU training. As IF and ECOD do not rely on GPU computation, they do not utilize VRAM. The comparison between ODIF and DIF clearly indicates significantly lower GPU memory usage for ODIF, with reductions exceeding 55% on average. This difference is supported by statistically significant p -values. An important observation is that ODIF maintains highly stable VRAM usage, unaffected by increases in the number of training samples. In contrast, DIF exhibits a sharp increase in GPU memory demands as the dataset size grows, posing a notable limitation for deployment on memory-constrained devices. Hence, large training sets can substantially raise the memory requirements for DIF. The final comparison, between ODIF and SGAE, reveals slightly lower VRAM usage for SGAE. However, both methods show stable memory behavior, without significant growth in VRAM consumption as dataset size increases. Memory usage is particularly important because, in the case of deploying the application, there may be device limitations on which it operates. Therefore, the stability of ODIF's memory usage is a crucial aspect.

Table 3

Train phase execution times for ODIF, DIF, IF, ECOD, and SGAE, measured separately for CPU and GPU stages. GPU results marked as “n/a” indicate that the method does not utilize GPU computations. p -values from statistical tests comparing each method with ODIF.

Dataset	Train time [ms]									
	CPU stage					GPU stage				
	ODIF	DIF	IF	ECOD	SGAE	ODIF	DIF	IF	ECOD	SGAE
ALOI	243 (28)	544 (80)	60 (7)	271 (35)	4457 (1600)	18 (2)	1299 (838)	n/a	n/a	8.6e4 (3.3e4)
Backdoor	589 (68)	1328 (234)	81 (4)	2853 (172)	1.6e4 (348)	26 (2)	8678 (553)	n/a	n/a	1.6e5 (2831)
Fault	1015 (97)	1170 (127)	127 (10)	13 (1)	150 (16)	16 (2)	102 (39)	n/a	n/a	2398 (278)
Fraud	318 (26)	2415 (476)	87 (7)	2531 (130)	2.0e4 (1240)	21 (2)	1.8e4 (156)	n/a	n/a	3.9e5 (2.5e4)
Glass	465 (48)	509 (58)	117 (10)	2 (0)	62 (9)	4 (1)	6 (3)	n/a	n/a	559 (88)
Ionosphere	970 (95)	1051 (128)	166 (17)	4 (0)	69 (12)	6 (1)	7 (3)	n/a	n/a	690 (117)
Landsat	998 (104)	1078 (109)	129 (12)	48 (5)	548 (64)	23 (10)	137 (86)	n/a	n/a	9483 (1225)
Letter	696 (71)	750 (105)	134 (14)	13 (3)	146 (17)	16 (2)	26 (8)	n/a	n/a	2165 (275)
Mnist	1093 (121)	1156 (116)	86 (7)	127 (15)	647 (128)	23 (3)	160 (99)	n/a	n/a	9632 (2003)
PageBlocks	525 (48)	582 (67)	80 (8)	13 (3)	365 (31)	21 (6)	93 (29)	n/a	n/a	6803 (621)
Satellite	498 (45)	555 (56)	128 (10)	49 (6)	502 (40)	22 (3)	100 (10)	n/a	n/a	8598 (787)
Satimage-2	463 (50)	515 (58)	120 (10)	46 (7)	441 (32)	21 (3)	90 (11)	n/a	n/a	7531 (635)
Skin	709 (76)	2327 (381)	134 (10)	144 (19)	1.5e4 (1447)	23 (3)	1.4e4 (312)	n/a	n/a	3.5e5 (3.5e4)
Smtip	445 (54)	987 (125)	90 (8)	66 (8)	6419 (617)	23 (3)	5110 (223)	n/a	n/a	1.4e5 (1.4e4)
Vowels	1062 (114)	1099 (131)	128 (11)	6 (1)	145 (17)	14 (2)	26 (15)	n/a	n/a	2207 (290)
Waveform	790 (72)	834 (83)	123 (13)	18 (2)	311 (26)	23 (15)	59 (29)	n/a	n/a	5278 (487)
Wilt	492 (61)	517 (51)	91 (9)	6 (1)	371 (56)	26 (16)	86 (27)	n/a	n/a	7235 (1120)
Wine	609 (64)	612 (54)	162 (15)	2 (0)	63 (9)	4 (2)	3 (0)	n/a	n/a	514 (73)
Average:	666 (69)	1002 (136)	114 (10)	345 (23)	3662 (317)	18 (4)	2646 (136)	n/a	n/a	6.7e4 (6591)
Median:	599 (66)	911 (107)	122 (10)	32 (4)	406 (36)	21 (2)	97 (29)	n/a	n/a	7383 (711)
p-value:	–	0.0000	0.0000	0.0304	0.8650	–	0.0000	n/a	n/a	0.0000

Table 4

Train phase memory usage for ODIF, DIF, IF, ECOD, and SGAE, measured for RAM and VRAM. VRAM results marked as “n/a” indicate that the method does not utilize VRAM. p -values from statistical tests comparing each method with ODIF.

Dataset	Train memory usage [MB]									
	RAM					VRAM				
	ODIF	DIF	IF	ECOD	SGAE	ODIF	DIF	IF	ECOD	SGAE
ALOI	738 (3)	917 (0)	558 (1)	661 (1)	1114 (0)	140 (0)	326 (0)	n/a	n/a	120 (0)
Backdoor	908 (7)	1251 (2)	792 (8)	1823 (7)	1413 (3)	140 (2)	506 (0)	n/a	n/a	128 (0)
Fault	728 (3)	733 (0)	544 (0)	582 (0)	1077 (0)	138 (1)	140 (0)	n/a	n/a	120 (0)
Fraud	796 (5)	1879 (4)	635 (11)	1100 (11)	1235 (1)	138 (0)	1270 (0)	n/a	n/a	120 (0)
Glass	719 (0)	719 (0)	542 (0)	576 (0)	1071 (0)	134 (0)	134 (0)	n/a	n/a	120 (0)
Ionosphere	726 (0)	725 (0)	544 (0)	577 (0)	1080 (0)	135 (1)	136 (0)	n/a	n/a	120 (0)
Landsat	732 (1)	745 (0)	545 (1)	593 (1)	1084 (0)	139 (1)	160 (0)	n/a	n/a	120 (0)
Letter	725 (1)	720 (0)	544 (0)	582 (1)	1080 (0)	137 (1)	140 (0)	n/a	n/a	120 (0)
Mnist	736 (3)	753 (0)	549 (0)	631 (0)	1092 (0)	140 (2)	166 (0)	n/a	n/a	124 (0)
PageBlocks	724 (3)	740 (0)	542 (0)	580 (0)	1076 (0)	139 (1)	154 (0)	n/a	n/a	120 (0)
Satellite	728 (2)	741 (0)	545 (1)	593 (1)	1080 (0)	139 (1)	160 (0)	n/a	n/a	120 (0)
Satimage-2	726 (2)	737 (1)	545 (0)	591 (1)	1080 (0)	139 (1)	158 (0)	n/a	n/a	122 (0)
Skin	737 (3)	1674 (0)	555 (0)	623 (1)	1114 (1)	138 (0)	1090 (0)	n/a	n/a	120 (0)
Smtip	728 (3)	1081 (0)	547 (0)	594 (1)	1094 (0)	139 (1)	506 (0)	n/a	n/a	120 (0)
Vowels	727 (3)	722 (0)	543 (0)	578 (0)	1079 (0)	137 (1)	140 (0)	n/a	n/a	120 (0)
Waveform	727 (2)	729 (0)	543 (0)	582 (0)	1080 (0)	139 (1)	148 (0)	n/a	n/a	120 (0)
Wilt	723 (2)	738 (1)	542 (0)	577 (0)	1079 (0)	139 (1)	152 (0)	n/a	n/a	120 (0)
Wine	719 (0)	719 (0)	543 (0)	576 (0)	1070 (0)	132 (0)	132 (0)	n/a	n/a	120 (0)
Average:	741 (2)	907 (1)	564 (1)	690 (1)	1111 (1)	138 (1)	312 (0)	n/a	n/a	121 (0)
Median:	727 (3)	739 (0)	544 (0)	587 (1)	1080 (0)	139 (1)	156 (0)	n/a	n/a	120 (0)
p-value:	–	0.0008	0.0000	0.0268	0.0000	–	0.0004	n/a	n/a	0.0000

6. Conclusions and future work

This paper introduces an optimized version of the Deep Isolation Forest algorithm, referred to as Optimized Deep Isolation Forest. The optimization of the training phase is achieved by introducing sampling before transforming the data into their representation spaces, thereby reducing the number of samples subjected to transformation. In addition, the mechanism for constructing isolation trees is modified by removing the sampling of elements during tree formation.

A comparative evaluation is conducted against four benchmark algorithms, including DIF and IF, as well as two non-IF-based techniques: ECOD and the deep learning-based method SGAE. A series of experiments is performed, in which the PR AUC performance metric is reported. The results demonstrate substantially better detection effectiveness of ODIF compared to IF, ECOD, and SGAE, while maintaining performance comparable to DIF. Moreover, execution times during the

training phase are analyzed separately for CPU and GPU operations. Memory usage, in terms of both RAM and VRAM consumption, is also examined. The experimental findings indicate considerable improvements across all measured aspects. These enhancements lead to reduced computational and memory complexity, thereby increasing the practical applicability of the ODIF algorithm, particularly in environments with limited computational resources.

Future research directions include the application of the representation mechanism and CERE for developing other anomaly detection methods, both those derived from Isolation Forest and others. Additionally, attention should be focused on the representation creation mechanism itself, where potentially training the network could be introduced to improve the method’s effectiveness. Another direction to consider is the preliminary preprocessing of data even before the transformation into representations, such as through clustering techniques.

CRediT authorship contribution statement

Łukasz Gałka: Visualization, Supervision, Resources, Methodology, Formal analysis, Conceptualization, Writing – original draft, Validation, Software, Project administration, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been supported by the internal grant FD-20/IT-3/047.

Data availability

Data will be made available on request.

References

- [1] A. Glyn-Davies, M. Girolami, Anomaly detection in streaming data with Gaussian process based stochastic differential equations, *Pattern Recognit.* 153 (2022) 254–260.
- [2] Z.-X. Guo, P.-L. Shui, Anomaly based sea-surface small target detection using k-nearest neighbor classification, *IEEE Trans. Aerosp. Electron. Syst.* 56 (6) (2020) 4947–4964.
- [3] W. Hu, J. Gao, B. Li, O. Wu, J. Du, S. Maybank, Anomaly detection using local kernel density estimation and context-based regression, *IEEE Trans. Knowl. Data Eng.* 32 (2) (2018) 218–233.
- [4] S. Sengupta, S. Das, Selective nearest neighbors clustering, *Pattern Recognit.* 155 (2022) 178–185.
- [5] R.A. Ariyaluran Habeeb, F. Nasaruddin, A. Gani, M.A. Amanullah, I. Abaker Targio Hashem, E. Ahmed, M. Imran, Clustering-based real-time anomaly detection—A breakthrough in big data technologies, *Trans. Emerg. Telecommun. Technol.* 33 (8) (2022) e3647.
- [6] R. Kromanis, P. Kripakaran, Support vector regression for anomaly detection from measurement histories, *Adv. Eng. Inf.* 27 (4) (2013) 486–495.
- [7] S.A. Aljawarneh, R. Vangipuram, GARUDA: Gaussian dissimilarity measure for feature representation and anomaly detection in Internet of things, *J. Supercomput.* 76 (6) (2020) 4376–4413.
- [8] J. Liu, H. Wang, H. Hang, S. Ma, X. Shen, Y. Shi, Self-supervised random forest on transformed distribution for anomaly detection, *IEEE Trans. Neural Netw. Learn. Syst.* (2024).
- [9] M. Peña, F. Biscarri, J.I. Guerrero, I. Monedero, C. León, Rule-based system to detect energy efficiency anomalies in smart buildings, a data mining approach, *Expert Syst. Appl.* 56 (2016) 242–255.
- [10] V. Carletti, P. Foggia, F. Rosa, M. Vento, Detecting malicious IoT network communication through Graph Neural Networks in real-world conditions, *Pattern Recognit.* 189 (2025) 133–139.
- [11] H. Lerogeron, R. Picot-Clément, A. Rakotomamonjy, L. Heutte, Approximating dynamic time warping with a convolutional neural network on EEG data, *Pattern Recognit.* 171 (2023) 162–169.
- [12] S. Fatemifar, S. Asadi, M. Awais, A. Akbari, J. Kittler, Face spoofing detection ensemble via multistage optimisation and pruning, *Pattern Recognit.* 158 (2022) 1–8.
- [13] H. Xu, G. Pang, Y. Wang, Y. Wang, Deep isolation forest for anomaly detection, *IEEE Trans. Knowl. Data Eng.* 35 (12) (2023) 12591–12604.
- [14] F.T. Liu, K.M. Ting, Z.-H. Zhou, Isolation forest, in: 2008 Eighth IEEE Int. Conf. Data Min., IEEE, 2008, pp. 413–422.
- [15] F.T. Liu, K.M. Ting, Z.-H. Zhou, Isolation-based anomaly detection, *ACM Trans. Knowl. Discov. Data* 6 (1) (2012) 1–39.
- [16] S. Hari, M.C. Kind, R.J. Brunner, Extended isolation forest, *IEEE Trans. Knowl. Data Eng.* 33 (4) (2019) 1479–1489.
- [17] Ł. Gałka, P. Karczmarek, M. Tokovarov, Effective enhancement of isolation forest method based on minimal spanning tree clustering, *Inf. Sci.* 628 (2023) 320–338.
- [18] S. Li, K. Zhang, P. Duan, X. Kang, Hyperspectral anomaly detection with kernel isolation forest, *IEEE Trans. Geosci. Remote Sens.* 58 (1) (2019) 319–329.
- [19] X. Tan, J. Yang, S. Rahardja, Sparse random projection isolation forest for outlier detection, *Pattern Recognit.* 163 (2022) 65–73.
- [20] S. Buschjäger, P.-J. Honysz, K. Morik, Randomized outlier detection with trees, *Int. J. Data Sci. Anal.* 13 (2) (2022) 91–104.
- [21] C. Shao, X. Du, J. Yu, J. Chen, Cluster-based improved isolation forest, *Entropy* 24 (5) (2022) 611.
- [22] Ł. Gałka, P. Karczmarek, M. Tokovarov, Isolation forest based on minimal spanning tree, *IEEE Access* 10 (2022) 74175–74186.
- [23] Ł. Gałka, P. Karczmarek, Minimal spanning tree-based isolation forest with anomaly score function built on the basis of fuzzy rules, *Appl. Soft Comput.* 148 (2023) 110935.
- [24] T. Liu, Z. Zhou, L. Yang, Layered isolation forest: A multi-level subspace algorithm for improving isolation forest, *Neurocomputing* 581 (2024) 127525.
- [25] A. Mensi, D.M. Tax, M. Bicego, Detecting outliers from pairwise proximities: Proximity isolation forests, *Pattern Recognit.* 138 (2023) 109334.
- [26] H. Xiang, X. Zhang, M. Dras, A. Beheshti, W. Dou, X. Xu, Deep optimal isolation forest with genetic algorithm for anomaly detection, in: 2023 IEEE Int. Conf. Data Min. ICDM, IEEE, 2023, pp. 678–687.
- [27] H. Xiang, Z. Salic, W. Dou, X. Xu, L. Qi, X. Zhang, OPHiForest: Order preserving hashing based isolation forest for robust and scalable anomaly detection, in: *Proc. 29th ACM Int. Conf. Inf. Knowl. Manag.*, 2020, pp. 1655–1664.
- [28] Ł. Gałka, P. Karczmarek, Deterministic attribute selection for isolation forest, *Pattern Recognit.* 151 (2024) 110395.
- [29] W.S. Al Farizi, I. Hidayah, M.N. Rizal, Isolation forest based anomaly detection: A systematic literature review, in: 2021 8th Int. Conf. Inf. Technol. Comput. Electr. Eng. ICITACEE, IEEE, 2021, pp. 118–122.
- [30] T. Barbariol, F.D. Chiara, D. Marcato, G.A. Susto, A review of tree-based approaches for anomaly detection, *Control. Charts Mach. Learn. Anom. Detect. Manuf.* (2022) 149–185.
- [31] Y. Song, S. Kuang, J. Huang, D. Zhang, Unsupervised anomaly detection of industrial building energy consumption, *Energy Built Env.* (2024).
- [32] M. Saremi, M. Bagheri, S.A. Agha Seyyed Mirzabozorg, N.E. Hassan, Z. Hoseinzade, A. Maghsoudi, S. Rezaia, H. Ranjbar, B. Zoheir, A. Beiranvand Pour, Evaluation of deep isolation forest (DIF) algorithm for mineral prospectivity mapping of polymetallic deposits, *Minerals* 14 (10) (2024) 1015.
- [33] D. Benka, D. Horváth, L. Špendla, G. Gašpar, M. Strémy, Machine learning-based detection of anomalies, intrusions and threats in industrial control systems, *IEEE Access* 13 (2025) 12502–12514.
- [34] X. Duan, H. Yan, D. Tian, J. Zhou, J. Su, W. Hao, In-vehicle CAN bus tampering attacks detection for connected and autonomous vehicles using an improved isolation forest method, *IEEE Trans. Intell. Transp. Syst.* 24 (2) (2021) 2122–2134.
- [35] M. Subramanian, K. Gokulakrishnan, B.V. Nirmala, S.V. Shankar, R.S. Krishnan, G.V. Rajkumar, GuardianForest: Cloud-powered patient health assurance through isolation forests, in: 2024 Int. Conf. Inven. Comput. Technol. ICICT, IEEE, 2024, pp. 2039–2046.
- [36] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, G.H. Chen, Ecod: Unsupervised outlier detection using empirical cumulative distribution functions, *IEEE Trans. Knowl. Data Eng.* 35 (12) (2023) 12181–12193.
- [37] Z. Huang, B. Zhang, G. Hu, L. Li, Y. Xu, Y. Jin, Enhancing unsupervised anomaly detection with score-guided network, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (10) (2024) 14754–14769.
- [38] S. Han, X. Hu, H. Huang, M. Jiang, Y. Zhao, Adbench: Anomaly detection benchmark, *Adv. Neural Inf. Process. Syst.* 35 (2022) 32142–32159.